AI Assisted Coding
Assignment Number: 11.

S. AKSHITH REDDY

2303A51383

BT.NO: 06

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Question

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support adding, searching, and deleting contacts.

Implement:

1. Contact Manager using arrays (lists)
2. Contact Manager using linked lists
   Perform:

   Add contact

   Search contact

   Delete contact
   Compare both approaches.

Python Code (Array-Based Implementation)

```
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append((name, phone))

    def search_contact(self, name):
        for contact in self.contacts:
            if contact[0] == name:
```

```
            return contact
        return None

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact[0] == name:
                self.contacts.remove(contact)
                return True
        return False



# Testing
manager = ContactManagerArray()
manager.add_contact("Ravi", "9876543210")
manager.add_contact("Anita", "9123456789")
print(manager.search_contact("Anita"))
manager.delete_contact("Ravi")
print(manager.contacts)
```

Output

```
('Anita', '9123456789')
[('Anita', '9123456789')]
```

Code Explanation

Contacts are stored using a Python list.

- Searching and deletion require linear traversal.
- Insertion is fast (O(1)), but deletion is slower (O(n)).
- This approach is simple but inefficient for frequent deletions.

Python Code (Linked List Implementation)

```
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node

    def search_contact(self, name):
        current = self.head
        while current:
            if current.name == name:
                return (current.name, current.phone)
            current = current.next
        return None

    def delete_contact(self, name):
        current = self.head
        prev = None
        while current:
```

```
            if current.name == name:
                if prev:
                    prev.next = current.next
                else:
                    self.head = current.next
                return True
            prev = current
            current = current.next
        return False



# Testing
manager = ContactManagerLinkedList()
manager.add_contact("Ravi", "9876543210")
manager.add_contact("Anita", "9123456789")
print(manager.search_contact("Ravi"))
manager.delete_contact("Anita")
print(manager.search_contact("Anita"))
```

Output

```
('Ravi', '9876543210')
None
```

Code Explanation

Linked list allows efficient insertion and deletion.

No shifting of elements is required.

Searching still takes O(n).

Better suited for dynamic data where frequent updates occur.

Comparison (Array vs Linked List)

Array insertion is faster but deletion is costly.

Linked list deletion is efficient but memory usage is higher.

Linked lists are preferred for dynamic datasets.

Task 2: Library Book Search System (Queues & Priority Queues)

Question

The SRU Library manages book requests where faculty requests must be prioritized over student requests.

Python Code (Queue Implementation)

```
from collections import deque


class BookQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if self.queue:
            return self.queue.popleft()
        return "Queue is empty"



# Testing
q = BookQueue()
q.enqueue("Student: Book A")
q.enqueue("Student: Book B")
```

```
print(q.dequeue())
```

Output

Student: Book A

Code Explanation

- Queue follows FIFO order.
- Suitable for simple request handling.
- Does not support prioritization.

Python Code (Priority Queue Implementation)

```
import heapq

class PriorityBookQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, priority, request):
        heapq.heappush(self.queue, (priority, request))

    def dequeue(self):
        if self.queue:
            return heapq.heappop(self.queue)[1]
        return "Queue is empty"


# Testing
pq = PriorityBookQueue()
pq.enqueue(2, "Student: Book A")
```

```
pq.enqueue(1, "Faculty: Book C")
print(pq.dequeue())
```

Output

Faculty: Book C

Code Explanation

Lower priority number means higher importance.

Faculty requests are processed before student requests.

Priority queues are ideal for scheduling systems.

Task 3: Emergency Help Desk (Stack Implementation)

Question

Implement a stack-based ticket system following LIFO behavior.

Python Code

```
class HelpDeskStack:
    def __init__(self):
        self.stack = []

    def push(self, ticket):
        self.stack.append(ticket)

    def pop(self):
        if self.stack:
            return self.stack.pop()
        return "No tickets"

    def peek(self):
```

```
        if self.stack:
            return self.stack[-1]
        return "No tickets"

    def is_empty(self):
        return len(self.stack) == 0



# Testing
desk = HelpDeskStack()
desk.push("Issue 1")
desk.push("Issue 2")
desk.push("Issue 3")
print(desk.pop())
print(desk.peek())
```

Output

Issue 3

Issue 2

Code Explanation

Stack follows Last-In, First-Out.

Useful for escalation systems.

Simple and efficient implementation.

Task 4: Hash Table with Collision Handling

Question

Implement a hash table using chaining.

Python Code

```
class HashTable:
    def __init__(self, size=10):
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % len(self.table)

    def insert(self, key, value):
        index = self._hash(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        index = self._hash(key)
        for pair in self.table[index]:
            if pair[0] == key:
                self.table[index].remove(pair)
                return True
```

```
        return False


# Testing
ht = HashTable()
ht.insert("ID101", "Ravi")
print(ht.search("ID101"))
ht.delete("ID101")
print(ht.search("ID101"))
```

Output

Ravi

None

Code Explanation

Chaining handles collisions.

Insert, search, and delete operate efficiently.

Hash tables provide average O(1) complexity.

Task 5: Real-Time Application Challenge

Question

Choose appropriate data structures and implement one feature.

Feature Mapping Table

Feature	**Data Structure**	**Justification**
Attendance Tracking	Array	Simple sequential storage
Event Registration	Queue	FIFO registration
Library Borrowing	Priority Queue	Faculty priority

Feature	Data Structure	Justification
Bus Scheduling	Graph	Route mapping
Cafeteria Orders	Queue	Order processing

Python Code (Cafeteria Order Queue)

```
from collections import deque

class CafeteriaQueue:
    def __init__(self):
        self.orders = deque()

    def place_order(self, order):
        self.orders.append(order)

    def serve_order(self):
        if self.orders:
            return self.orders.popleft()
        return "No orders"



# Testing
cafeteria = CafeteriaQueue()
cafeteria.place_order("Order 1")
cafeteria.place_order("Order 2")
print(cafeteria.serve_order())
```

Output

Order 1

Code Explanation

Queue ensures fair order processing.

FIFO suits real-time service systems.

AI-assisted generation improved clarity and correctness.